

Market Screener

Multi-Source Stock Screener with Airflow Pipeline and AI Analysis
Technical Foundations and Architecture

Doctum Consilium
Generated 2026-05-08

Abstract

Automated stock screener with intraday and nightly Airflow pipelines, FastAPI backend, Yahoo Finance and Alpaca connectors, and AI-powered analysis. This document covers the theoretical foundations, architectural decisions, and key technical concepts required to understand, contribute to, and maintain this repository.

Contents

Overview

Market Screener automates the identification of trading opportunities by running configurable screening rules across a universe of instruments. Two Airflow pipelines run on distinct schedules: intraday (real-time signals) and nightly (batch scoring and database refresh).

The backend exposes a REST API for queried results, and the AI analysis module generates natural-language summaries of screener output.

Architecture

Airflow Scheduler

```

Intraday DAG → Yahoo Finance / Alpaca fetch
               → Apply screening rules
               → Write to PostgreSQL

Nightly DAG   → Historical scoring
               → AI analysis (LiteLLM)
               → Dashboard refresh
  
```

FastAPI backend PostgreSQL React frontend

Key Technical Concepts

- Technical indicators: $RSI = 100 - \frac{100}{1+RS}$ where $RS = \frac{avggain}{avgloss}$; $MACD = EMA_{12} - EMA_{26}$; Bollinger Bands = $\mu \pm 2\sigma$ over rolling window.
- Airflow DAG design: idempotent tasks, catchup=False for live data, task retries with exponential backoff.
- Database schema: time-series table with (symbol, timestamp, OHLCV, indicators); partitioned by date for query performance.
- Screening rules: configurable filter chains; e.g. $RSI < 30$ AND price > 50-day MA AND volume spike.

Data Flow

Airflow DAG trigger → Yahoo Finance REST → OHLCV normalisation → Technical indicator calculation → Screening rule application → PostgreSQL upsert → FastAPI read → React display.

Technology Stack

Component	Technology
Orchestration	Apache Airflow 2.x
Data sources	yfinance, Alpaca Market Data API
Indicators	pandas-ta, TA-Lib
Database	PostgreSQL
API	FastAPI
AI analysis	LiteLLM

Design Decisions

- Two separate DAGs (intraday vs. nightly) to isolate latency-sensitive real-time paths from batch operations.
- yfinance as primary source (free); Alpaca as fallback for real-time data requiring an API key.

Repository Structure

Listing 1: Top-level directory layout

```
market_screener/  
  .github/  
    copilot-instructions.md    # GitHub Copilot guardrails  
    agents/  
      ecr-secrets-agent.agent.md # ECR secret rotation runbook  
  CLAUDE.md                   # Claude Code guardrails  
  GEMINI.md                   # Gemini CLI guardrails  
  INFRASTRUCTURE.md           # Operational runbook  
  ROADMAP.md                  # Execution log and roadmap  
  README.md                   # Project overview
```

Contributing

1. Read README.md, INFRASTRUCTURE.md, and ROADMAP.md before any task.
2. Follow the Code Documentation Standard: every public function must have a docstring (Google-style for Python, JSDoc for TypeScript, XML for C#).
3. Run the guardrails validator before committing:

```
git add -A && ./guardrails/bin/validate_guardrails.sh
```

4. For deployment or destructive operations, always use a named tmux session:

```
tmux new-session -A -s deploy-market_screener
```